

DEPARTAMENTO:	Ciencias de la Computación	CARRERA:	Tecnología de la Información		
ASIGNATURA:	Fundamentos de sistemas web	NIVEL:	4to	FECHA:	19/07/2026
DOCENTE:	Ing. Geovanny Brito	PRÁCTICA N°:	3.2	CALIFICACIÓN:	

Registro de estudiantes con manipulación del DOM: CRUD, modal JSON, búsqueda, filtros y librerías externas

Daniel Gustavo Barragan Montero

Enlace del repositorio GitHub:

<https://github.com/dgbarragan-lang/laboratorio-js-dom.git>

RESUMEN

En esta práctica se amplió el registro de estudiantes mediante manipulación del DOM en JavaScript puro, partiendo de los archivos base index.html y app.js. Se implementó un arreglo global de objetos para almacenar los estudiantes, una tabla dinámica generada a partir de dicho arreglo, una columna de acciones con botones para consultar, editar y eliminar cada registro, y un modal de Bootstrap que permite visualizar la información en formato JSON, tanto de un estudiante individual como del listado completo. Se agregaron además funcionalidades de edición y eliminación de registros (esta última con confirmación mediante SweetAlert2), un contador dinámico de estudiantes, un mecanismo de limpieza general, mensajes visuales con la librería Toastify, y dos herramientas de consulta sobre el arreglo: una búsqueda por texto libre y un filtro por carrera. El sistema fue probado con un mínimo de cinco estudiantes, verificando el correcto funcionamiento de cada una de las operaciones. Como resultado se obtuvo una aplicación funcional que refuerza el acceso y la modificación de elementos HTML, el manejo de eventos, la creación dinámica de nodos y el trabajo con arreglos de objetos y datos en formato JSON.

Palabras clave: DOM, JavaScript, Bootstrap.

ÍNDICE DE CONTENIDO

1. INTRODUCCIÓN	4
2. OBJETIVOS	4
3. MARCO TEÓRICO	4
4. DESCRIPCIÓN DEL PROCEDIMIENTO	4
4.1 Arreglo global de estudiantes y registro de datos	4
4.2 Columna de acciones en la tabla	6
4.3 Modal de Bootstrap para visualizar JSON individual	7
4.4 JSON general de todos los estudiantes	8
4.5 Edición de estudiantes	10
4.6 Eliminación de estudiantes con confirmación	12
4.7 Contador de estudiantes	14
4.8 Limpieza general de tabla, arreglo y formulario	15
4.9 Mensajes visuales con Toastify	16
4.10 Búsqueda de estudiantes	17
4.11 Filtro por carrera	18
4.12 Prueba integral con cinco estudiantes	19
5. LIBRERÍA EXTERNA UTILIZADA	20
6. ANÁLISIS DE RESULTADOS	21
7. DISCUSIÓN	21
8. CONCLUSIONES Y CRITERIO PERSONAL	21
9. BIBLIOGRAFÍA	21

ÍNDICE DE FIGURAS

Formulario de registro y tabla con estudiantes almacenados en el arreglo.	6
Columna de acciones con los botones Ver JSON, Editar y Eliminar en cada fila.	7
Modal de Bootstrap mostrando el JSON de un estudiante individual.	8
Modal mostrando el JSON general con todos los estudiantes registrados.	9
Mensaje mostrado en el modal cuando aún no existen estudiantes registrados.	10
Formulario cargado en modo edición, con el botón indicando "Guardar cambios".	11
Tabla actualizada tras guardar los cambios de un estudiante editado.	12
Cuadro de confirmación de SweetAlert2 antes de eliminar un estudiante.	13
Tabla actualizada después de confirmar la eliminación de un registro.	14
Contador de estudiantes reflejando la cantidad real de registros.	15
Confirmación y resultado de la limpieza general del sistema.	16
Notificación Toastify mostrada al agregar un estudiante.	17
Resultado de la tabla al buscar por nombre, carrera o semestre.	18
Tabla filtrada mostrando únicamente a los estudiantes de la carrera seleccionada.	19
Tabla con al menos cinco estudiantes registrados, utilizada para las pruebas integrales del sistema.	20

1. INTRODUCCIÓN

La presente práctica corresponde a la asignatura Fundamentos Web y tiene como propósito reforzar los conocimientos sobre manipulación del Document Object Model (DOM) mediante JavaScript. Partiendo del registro de estudiantes construido en clase (formulario con nombre, carrera y semestre, tabla dinámica, contador de estudiantes y funciones básicas de obtención de datos, validación, limpieza y despliegue de mensajes) se amplía el sistema incorporando nuevas funcionalidades: presentación de la información en formato JSON dentro de un modal de Bootstrap, edición y eliminación de registros, búsqueda y filtrado sobre el arreglo de datos, y la integración de una librería externa (SweetAlert2 y/o Toastify) para mejorar la interacción visual con el usuario. El desarrollo de la práctica se realizó siguiendo las normas de organización, disciplina y buenas prácticas de programación indicadas por el docente.

2. OBJETIVOS

Ampliar el sistema de registro de estudiantes desarrollado en clase aplicando manipulación del DOM, trabajo con arreglos de objetos, presentación de datos en un modal de Bootstrap en formato JSON, edición y eliminación de registros, mecanismos de búsqueda y filtrado, y la integración de una librería externa que mejore los mensajes visuales de la aplicación.

3. MARCO TEÓRICO

El DOM (Document Object Model) es una representación estructurada del documento HTML en forma de árbol de objetos. Cuando el navegador carga una página, convierte cada una de sus etiquetas en objetos que JavaScript puede leer, modificar, crear o eliminar. Gracias al DOM, un programa puede cambiar textos, agregar filas a una tabla, mostrar mensajes, abrir componentes, modificar clases de Bootstrap y actualizar información en pantalla sin necesidad de recargar la página.

Un concepto central en la manipulación del DOM es el manejo de eventos, en especial el evento click, ya que permite ejecutar una función determinada cada vez que el usuario presiona un botón. En este proyecto se utiliza además la delegación de eventos: en lugar de asignar un listener a cada botón de la tabla, se coloca un único listener en el elemento contenedor (tbody) que identifica, mediante atributos data-*, qué acción y sobre qué registro debe ejecutarse.

Entre los métodos y propiedades del DOM utilizados en la práctica se encuentran: getElementById(), para obtener la referencia a un elemento a partir de su identificador; createElement(), para crear nuevos nodos (por ejemplo filas <tr> y celdas <td>); appendChild(), para insertar esos nodos dentro del documento; textContent, para asignar texto plano a un elemento; innerHTML, para insertar marcado HTML (por ejemplo los botones de acciones dentro de una celda); className y classList, para leer o modificar las clases CSS/Bootstrap de un elemento; y JSON.stringify(), que convierte un objeto o arreglo de JavaScript en una cadena de texto en formato JSON, utilizada aquí para mostrar la información de los estudiantes dentro del modal.

4. DESCRIPCIÓN DEL PROCEDIMIENTO

A continuación se documenta cada una de las funcionalidades agregadas sobre la base construida en clase (index.html y app.js). Para cada apartado se incluye una breve explicación, el código HTML y/o JavaScript relevante, la captura correspondiente al resultado obtenido y una descripción de la prueba realizada.

4.1 Arreglo global de estudiantes y registro de datos

Se creó un arreglo global (estudiantes) que actúa como fuente principal de datos del sistema, junto con una variable editIndex para controlar el modo de edición. Cada vez que el formulario se envía correctamente, los datos ingresados

(nombre, carrera y semestre) se almacenan como un objeto dentro de dicho arreglo, y la tabla se vuelve a dibujar (renderTabla) leyendo su contenido directamente desde el arreglo.

Código HTML (formulario y tabla):

```
1. <div class="col-md-4">
2.   <label for="nombre" class="form-label">Nombre</label>
3.   <input id="nombre" type="text" class="form-control"
4.     placeholder="Ejemplo: Ana Torres" />
5. </div>
6. <div class="col-md-4">
7.   <label for="carrera" class="form-label">Carrera</label>
8.   <select id="carrera" class="form-select">
9.     <option value="">Seleccione...</option>
10.    <option value="Tecnologías de la Información">Tecnologías de la Información</option>
11.    <option value="Software">Software</option>
12.    <option value="Sistemas">Sistemas</option>
13.  </select>
14. </div>
15. <div class="col-md-4">
16.   <label for="semestre" class="form-label">Semestre</label>
17.   <input id="semestre" type="number" class="form-control" min="1" max="10" />
18. </div>
19. <button id="btnAgregar" class="btn btn-primary">Agregar estudiante</button>
20.
21. <table class="table table-striped table-bordered mt-3">
22.   <thead class="table-dark">
23.     <tr><th>#</th><th>Nombre</th><th>Carrera</th><th>Semestre</th><th>Acciones</th></tr>
24.   </thead>
25.   <tbody id="cuerpoTabla"></tbody>
26. </table>
```

Código JavaScript (arreglo global y registro):

```
1. let estudiantes = [];
2. let editIndex = -1;
3.
4. function manejarAgregar() {
5.   const estudiante = obtenerDatosFormulario();
6.   if (!validarEstudiante(estudiante)) {
7.     mostrarToast("Complete todos los campos.", "warning");
8.     return;
9.   }
10.  if (editIndex >= 0) {
11.    estudiantes[editIndex] = estudiante;
12.    editIndex = -1;
13.    btnAgregar.textContent = "Agregar estudiante";
14.    mostrarToast("Estudiante editado correctamente.", "success");
15.  } else {
16.    estudiantes.push(estudiante);
17.    mostrarToast("Estudiante agregado correctamente.", "success");
18.  }
19.  renderTabla();
20.  limpiarFormulario();
21. }
22.
23. function obtenerDatosFormulario() {
24.   return {
25.     nombre: nombreInput.value.trim(),
26.     carrera: carreraInput.value,
27.     semestre: semestreInput.value.trim()
28.   };
29. }
30.
31. function validarEstudiante(est) {
32.   return est.nombre !== "" && est.carrera !== "" && est.semestre !== "";
```

Registro de estudiantes

Ingrese los datos del estudiante y agréguelo dinámicamente a la tabla.

Nombre

Carrera

Semestre

Agregar estudiante
Limpiar tabla
Ver JSON general

Total de estudiantes registrados: **1**

Buscar
Limpiar búsqueda

Filtrar

#	Nombre	Carrera	Semestre	Acciones
1	Daniel Barragan	Tecnologías de la Información	4	Ver JSON Editar Eliminar

Figura 1. Formulario de registro y tabla con estudiantes almacenados en el arreglo.

Prueba realizada: se completó el formulario con datos válidos y se presionó "Agregar estudiante", verificando que el nuevo registro se almacenara en el arreglo y apareciera inmediatamente en la tabla.

4.2 Columna de acciones en la tabla

Se agregó una columna "Acciones" en la tabla, generada dinámicamente para cada fila mediante innerHTML, con tres botones (Ver JSON, Editar, Eliminar), cada uno identificado con atributos data-action y data-index. En lugar de asignar un listener por botón, se utiliza delegación de eventos sobre el elemento cuerpoTabla, que identifica la acción solicitada y el índice del estudiante correspondiente.

Código HTML (contenedor de la tabla):

```
1. <tbody id="cuerpoTabla"></tbody>
```

Código JavaScript (columna de acciones y delegación de eventos):

```
1. const tdAcciones = document.createElement("td");
2. tdAcciones.innerHTML = `
3.   <button class="btn btn-sm btn-info me-1" data-action="ver" data-index="${i}">Ver JSON</button>
4.   <button class="btn btn-sm btn-warning me-1" data-action="editar" data-index="${i}">Editar</button>
5.   <button class="btn btn-sm btn-danger" data-action="eliminar" data-index="${i}">Eliminar</button>
6. `;
7.
8. cuerpoTabla.addEventListener("click", (e) => {
9.   const target = e.target;
10.  const accion = target.dataset.action;
11.  const index = target.dataset.index;
12.  if (!accion) return;
13.
14.  if (accion === "ver") {
15.    mostrarJsonIndividual(parseInt(index, 10));
16.  } else if (accion === "editar") {
17.    iniciarEdicion(parseInt(index, 10));
18.  } else if (accion === "eliminar") {
19.    eliminarEstudiante(parseInt(index, 10));
20.  }
21. });
```

#	Nombre	Carrera	Semestre	Acciones
1	Daniel Barragan	Tecnologías de la Información	4	Ver JSON Editar Eliminar

Figura 2. Columna de acciones con los botones Ver JSON, Editar y Eliminar en cada fila.

Prueba realizada: se verificó que cada fila de la tabla mostrara los tres botones de acción y que al presionarlos se ejecutara la función correspondiente según el data-index de la fila.

4.3 Modal de Bootstrap para visualizar JSON individual

Se creó un modal de Bootstrap (jsonModal) que se abre desde JavaScript y muestra la información de un estudiante en formato JSON, generado mediante JSON.stringify(data, null, 2) para una lectura ordenada. El botón "Ver JSON" de cada fila invoca mostrarJsonIndividual(index), que obtiene el estudiante correspondiente del arreglo y llama a mostrarJsonEnModal() para desplegarlo.

Código HTML (modal):

```
1. <div class="modal fade" id="jsonModal" tabindex="-1" aria-hidden="true">
2.   <div class="modal-dialog modal-lg modal-dialog-centered">
3.     <div class="modal-content">
```

```

4.     <div class="modal-header">
5.         <h5 class="modal-title">JSON del estudiante</h5>
6.         <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
7.     </div>
8.     <div class="modal-body">
9.         <pre id="modalJsonContent" class="small"></pre>
10.    </div>
11.    <div class="modal-footer">
12.        <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Cerrar</button>
13.    </div>
14. </div>
15. </div>
16. </div>

```

Código JavaScript (JSON individual y apertura del modal):

```

1. function mostrarJsonIndividual(index) {
2.     const est = estudiantes[index];
3.     if (!est) return mostrarToast("Estudiante no encontrado.", "error");
4.     mostrarJsonEnModal(est, "JSON estudiante");
5. }
6.
7. function mostrarJsonEnModal(data, titulo = "JSON") {
8.     const modalEl = document.getElementById("jsonModal");
9.     const contenido = document.getElementById("modalJsonContent");
10.    const modalTitle = modalEl.querySelector(".modal-title");
11.    modalTitle.textContent = titulo;
12.    contenido.textContent = JSON.stringify(data, null, 2);
13.    const modal = new bootstrap.Modal(modalEl);
14.    modal.show();
15. }

```



Figura 3. Modal de Bootstrap mostrando el JSON de un estudiante individual.

Prueba realizada: se presionó el botón "Ver JSON" de distintas filas, comprobando que el modal se abriera mostrando únicamente los datos del estudiante seleccionado, correctamente formateados.

4.4 JSON general de todos los estudiantes

Se agregó el botón "Ver JSON general", que invoca mostrarJsonGeneral(). Esta función reutiliza el mismo modal para mostrar el arreglo completo de estudiantes en formato JSON. Se validó además que, si el arreglo se encuentra vacío, el modal muestre un mensaje indicando que no existen estudiantes registrados en lugar de un arreglo vacío.

Código HTML (botón):

```
1. <button id="btnVerJsonGeneral" class="btn btn-info text-white">Ver JSON general</button>
```

Código JavaScript (JSON general):

```
1. function mostrarJsonGeneral() {  
2.   if (estudiantes.length === 0) {  
3.     mostrarJsonEnModal({ mensaje: "No existen estudiantes registrados." }, "JSON general");  
4.     return;  
5.   }  
6.   mostrarJsonEnModal(estudiantes, "JSON general");  
7. }  
8.  
9. btnVerJsonGeneral.addEventListener("click", mostrarJsonGeneral);
```

JSON general

```
[  
  {  
    "nombre": "Daniel Barragan",  
    "carrera": "Tecnologías de la Información",  
    "semestre": "4"  
  },  
  {  
    "nombre": "Lucia Chisaguano",  
    "carrera": "Tecnologías de la Información",  
    "semestre": "4"  
  }  
]
```

Cerrar

1	Daniel Barragan	Tecnologías de la Información	4	Ver JSON Editar Eliminar
2	Lucia Chisaguano	Tecnologías de la Información	4	Ver JSON Editar Eliminar

Figura 4. Modal mostrando el JSON general con todos los estudiantes registrados.

Complete todos los campos. ✕

Registro de estudiantes

Ingrese los datos del estudiante y agréguelo dinámicamente a la tabla.

Nombre

Carrera

Seleccione...
▼

Semestre

Agregar estudiante

Limpiar tabla

Ver JSON general

Total de estudiantes registrados: 0

Buscar

Limpiar búsqueda

Todas
▼

Filtrar

#	Nombre	Carrera	Semestre	Acciones
---	--------	---------	----------	----------

Figura 5. Mensaje mostrado en el modal cuando aún no existen estudiantes registrados.

Prueba realizada: se consultó el JSON general con la tabla vacía (mostrando el mensaje de aviso) y luego con varios estudiantes registrados, verificando que el arreglo completo se mostrara correctamente.

4.5 Edición de estudiantes

Al presionar "Editar" en una fila, se ejecuta `iniciarEdicion(index)`, que carga los datos del estudiante seleccionado en el formulario y guarda su posición en la variable `editIndex`, activando el modo edición. Mientras este modo está activo, el texto del botón principal cambia a "Guardar cambios"; al confirmar, `manejarAgregar()` detecta que `editIndex` es distinto de -1 y sobrescribe el objeto correspondiente en el arreglo en lugar de crear uno nuevo, actualizando después la tabla y regresando el botón a su estado inicial.

Código JavaScript (modo edición):

```

1. function iniciarEdicion(index) {
2.   const est = estudiantes[index];
3.   if (!est) return mostrarToast("Estudiante no encontrado.", "error");
4.   nombreInput.value = est.nombre;

```

```
5. carreraInput.value = est.carrera;
6. semestreInput.value = est.semestre;
7. editIndex = index;
8. btnAgregar.textContent = "Guardar cambios";
9. }
10.
11. // Dentro de manejarAgregar():
12. if (editIndex >= 0) {
13.   estudiantes[editIndex] = estudiante;
14.   editIndex = -1;
15.   btnAgregar.textContent = "Agregar estudiante";
16.   mostrarToast("Estudiante editado correctamente.", "success");
17. }
```

Registro de estudiantes

Ingrese los datos del estudiante y agréguelo dinámicamente a la tabla.

Nombre

Carrera

Semestre

Guardar cambiosLimpiar tablaVer JSON general

Total de estudiantes registrados: 1

BuscarLimpiar búsquedaFiltrar

#	Nombre	Carrera	Semestre	Acciones
1	Daniel Barragan	Tecnologías de la Información	4	<button>Ver JSON</button> <button>Editar</button> <button>Eliminar</button>

Figura 6. Formulario cargado en modo edición, con el botón indicando "Guardar cambios".

Estudiante editado correctamente.

Registro de estudiantes

Ingrese los datos del estudiante y agréguelo dinámicamente a la tabla.

Nombre

Carrera

Semestre

Agregar estudiante
Limpiar tabla
Ver JSON general

Total de estudiantes registrados: **1**

Buscar
Limpiar búsqueda

Todas
Filtrar

#	Nombre	Carrera	Semestre	Acciones
1	Daniel Gustavo Barragan Montero	Tecnologías de la Información	4	Ver JSON Editar Eliminar

Figura 7. Tabla actualizada tras guardar los cambios de un estudiante editado.

Prueba realizada: se editó un estudiante existente, confirmando que sus datos se cargaran en el formulario, que el botón cambiara de texto y que, al guardar, la tabla y el arreglo reflejaran la información actualizada.

4.6 Eliminación de estudiantes con confirmación

Se agregó la opción de eliminar cada registro. Antes de borrar un estudiante, se solicita confirmación mediante SweetAlert2, evitando eliminaciones accidentales. Si el usuario confirma, el registro se elimina del arreglo con splice() y la tabla se vuelve a dibujar.

Código JavaScript (eliminación con confirmación):

```

1. function eliminarEstudiante(index) {
2.   const est = estudiantes[index];
3.   if (!est) return mostrarToast("Estudiante no encontrado.", "error");
4.
5.   Swal.fire({
6.     title: "¿Eliminar estudiante?",
7.     text: `Se eliminará a ${est.nombre}. Esta acción no se puede deshacer.`,
8.     icon: "warning",
9.     showCancelButton: true,
10.    confirmButtonText: "Sí, eliminar",
11.    cancelButtonText: "Cancelar"

```

```

12. }).then((result) => {
13.   if (result.isConfirmed) {
14.     estudiantes.splice(index, 1);
15.     renderTabla();
16.     mostrarToast("Estudiante eliminado.", "success");
17.   }
18. });
19. }

```



¿Eliminar estudiante?

Se eliminará a Daniel Gustavo Barragan Montero. Esta acción no se puede deshacer.

Sí, eliminar

Cancelar

#	Nombre	Carrera	Semestre	Acciones
1	Daniel Gustavo Barragan Montero	Tecnologías de la Información	4	Ver JSON Editar Eliminar

Figura 8. Cuadro de confirmación de SweetAlert2 antes de eliminar un estudiante.

Estudiante eliminado. ✕

Registro de estudiantes

Ingrese los datos del estudiante y agréguelo dinámicamente a la tabla.

Nombre

Carrera

Semestre

Agregar estudiante
Limpiar tabla
Ver JSON general

Total de estudiantes registrados: **0**

Buscar
Limpiar búsqueda

Todas
Filtrar

#	Nombre	Carrera	Semestre	Acciones
---	--------	---------	----------	----------

Figura 9. Tabla actualizada después de confirmar la eliminación de un registro.

Prueba realizada: se intentó eliminar un estudiante, se canceló la primera vez para comprobar que el registro no se eliminaba, y luego se confirmó, verificando que desapareciera de la tabla y del arreglo.

4.7 Contador de estudiantes

El contador de estudiantes se actualiza mediante la función `actualizarTotal()`, invocada al final de `renderTabla()`. Esta función refleja siempre la cantidad real de elementos del arreglo `estudiantes`, por lo que se mantiene sincronizado luego de agregar, editar, eliminar o limpiar registros.

Código HTML (contador):

```
1. <p>Total de estudiantes registrados: <strong id="totalEstudiantes">0</strong></p>
```

Código JavaScript (actualización del contador):

```
1. function actualizarTotal() {
2.   totalEstudiantes.textContent = estudiantes.length;
3. }
```

Agregar estudiante
Limpiar tabla
Ver JSON general

Total de estudiantes registrados: **2**

Buscar
Limpiar búsqueda

Todas
Filtrar

#	Nombre	Carrera	Semestre	Acciones
1	Daniel Barragan	Tecnologías de la Información	4	Ver JSON Editar Eliminar
2	Lucia Chisaguano	Tecnologías de la Información	4	Ver JSON Editar Eliminar

Figura 10. Contador de estudiantes reflejando la cantidad real de registros.

Prueba realizada: se comprobó que el contador aumentara al agregar estudiantes, se mantuviera correcto tras editar, y disminuyera al eliminar o limpiar registros.

4.8 Limpieza general de tabla, arreglo y formulario

El botón "Limpiar tabla" solicita confirmación mediante SweetAlert2 y, de ser aceptada, reinicia el arreglo de estudiantes, el índice de edición, la tabla, el contador y el botón principal, dejando el sistema en su estado inicial.

Código JavaScript (limpieza general):

```

1. function limpiarTodo() {
2.   Swal.fire({
3.     title: "¿Limpiar todo?",
4.     text: "Se eliminarán todos los registros y se reiniciará el contador.",
5.     icon: "warning",
6.     showCancelButton: true,
7.     confirmButtonText: "Sí, limpiar",
8.     cancelButtonText: "Cancelar"
9.   }).then((result) => {
10.    if (result.isConfirmed) {
11.      estudiantes = [];
12.      editIndex = -1;
13.      btnAgregar.textContent = "Agregar estudiante";
14.      renderTabla();
15.      mostrarToast("Datos limpiados correctamente.", "info");
16.    }
17.  });
18. }

```

Registro de estudiantes

Ingrese los datos del estudiante y agréguelo dinámicamente a la tabla.

Nombre

Steven Fernando

Carrera



¿Limpiar todo?

Se eliminarán todos los registros y se reiniciará el contador.

Sí, limpiar

Cancelar

Figura 11. Confirmación y resultado de la limpieza general del sistema.

Prueba realizada: con varios estudiantes registrados, se presionó "Limpiar tabla", se confirmó la acción y se verificó que la tabla, el arreglo, el formulario y el contador volvieran a su estado inicial.

4.9 Mensajes visuales con Toastify

La función `mostrarToast()` centraliza la presentación de notificaciones breves mediante la librería `Toastify`, utilizadas para informar al usuario cuando un estudiante es agregado, editado, eliminado, cuando se limpia la información o cuando existe una validación pendiente en el formulario.

Código JavaScript (notificaciones Toastify):

```
1. function mostrarToast(texto, tipo = "info") {
2.   let background = "#3498db";
3.   if (tipo === "success") background = "#28a745";
4.   if (tipo === "warning") background = "#f39c12";
5.   if (tipo === "error") background = "#dc3545";
6.   if (tipo === "info") background = "#6c757d";
7.
8.   Toastify({
9.     text: texto,
10.    duration: 2500,
11.    close: true,
12.    gravity: "top",
13.    position: "right",
14.    style: { background }
15.  }).showToast();
```

Estudiante agregado correctamente. ✕

Registro de estudiantes

Ingrese los datos del estudiante y agréguelo dinámicamente a la tabla.

Nombre

Carrera

Seleccione...
▼

Semestre

Agregar estudiante
Limpiar tabla
Ver JSON general

Total de estudiantes registrados: **1**

Buscar
Limpiar búsqueda

Todas
▼

Filtrar

#	Nombre	Carrera	Semestre	Acciones
1	Steven Fernando	Software	3	Ver JSON Editar Eliminar

Figura 12. Notificación Toastify mostrada al agregar un estudiante.

Prueba realizada: se generaron distintos tipos de notificación (éxito, advertencia, información) para comprobar que el color y el mensaje mostrado correspondieran a cada acción realizada.

4.10 Búsqueda de estudiantes

Se agregó un input de búsqueda y un botón "Buscar". El valor ingresado se captura con `inputBuscar.value`, se convierte a minúsculas y se compara contra el nombre, la carrera y el semestre de cada estudiante del arreglo mediante `filter()` e `includes()`. El resultado filtrado se pasa como parámetro a `renderTabla()`, que vuelve a dibujar la tabla únicamente con las coincidencias encontradas. El botón "Limpiar búsqueda" vacía el input y vuelve a mostrar todos los estudiantes.

Código HTML (input y botones de búsqueda):

```

1. <input id="inputBuscar" type="text" class="form-control"
2.   placeholder="Buscar por nombre, carrera o semestre" />
3. <button id="btnBuscar" class="btn btn-outline-primary">Buscar</button>
4. <button id="btnLimpiarBusqueda" class="btn btn-outline-secondary">Limpiar búsqueda</button>

```

Código JavaScript (filtrado por texto sobre el arreglo):

```

1. function buscarEstudiantes() {
2.   const texto = inputBuscar.value.trim().toLowerCase();
3.   if (texto === "") return renderTabla();

```

```

4.
5.   const resultados = estudiantes.filter((e) => {
6.     return (
7.       e.nombre.toLowerCase().includes(texto) ||
8.       e.carrera.toLowerCase().includes(texto) ||
9.       String(e.semestre).toLowerCase().includes(texto)
10.    );
11.  });
12.
13.  renderTabla(resultados);
14. }
15.
16. btnBuscar.addEventListener("click", buscarEstudiantes);
17. btnLimpiarBusqueda.addEventListener("click", () => {
18.   inputBuscar.value = "";
19.   renderTabla();
20. });

```

Registro de estudiantes

Ingrese los datos del estudiante y agréguelo dinámicamente a la tabla.

Nombre

Carrera

Selecione...

Semestre

Agregar estudiante
Limpiar tabla
Ver JSON general

Total de estudiantes registrados: 2

Buscar
Limpiar búsqueda

Todas

Filtrar

#	Nombre	Carrera	Semestre	Acciones
1	Steven barragan	Software	3	Ver JSON Editar Eliminar

Figura 13. Resultado de la tabla al buscar por nombre, carrera o semestre.

Prueba realizada: se buscaron estudiantes por nombre parcial, por carrera y por semestre, verificando que la tabla mostrara solo las coincidencias, y que "Limpiar búsqueda" restableciera el listado completo.

4.11 Filtro por carrera

Se agregó un select adicional (filtroCarrera) con las carreras disponibles y un botón "Filtrar". Al presionarlo, se toma el valor seleccionado y se compara con la propiedad carrera de cada estudiante del arreglo mediante filter(); el

resultado se envía nuevamente a renderTabla(). Si se selecciona la opción "Todas", se vuelve a mostrar el listado completo de estudiantes.

Código HTML (select y botón de filtro):

```
1. <select id="filtroCarrera" class="form-select">
2.   <option value="Todas">Todas</option>
3.   <option value="Tecnologías de la Información">Tecnologías de la Información</option>
4.   <option value="Software">Software</option>
5.   <option value="Sistemas">Sistemas</option>
6. </select>
7. <button id="btnFiltrar" class="btn btn-outline-primary">Filtrar</button>
```

Código JavaScript (filtrado por carrera sobre el arreglo):

```
1. function aplicarFiltroCarrera() {
2.   const seleccionado = filtroCarrera.value;
3.   if (seleccionado === "Todas") return renderTabla();
4.   const resultados = estudiantes.filter((e) => e.carrera === seleccionado);
5.   renderTabla(resultados);
6. }
7.
8. btnFiltrar.addEventListener("click", aplicarFiltroCarrera);
```

Total de estudiantes registrados: 2

Buscar por nombre, carrera o sem

Buscar

Limpiar búsqueda

Tecnologías de la Información

Filtrar

#	Nombre	Carrera	Semestre	Acciones
1	Daniel Barragan	Tecnologías de la Información	4	Ver JSON Editar Eliminar

Figura 14. Tabla filtrada mostrando únicamente a los estudiantes de la carrera seleccionada.

Prueba realizada: se filtró por cada una de las carreras disponibles y se comprobó que solo se mostraran los estudiantes correspondientes; al seleccionar "Todas" se restableció el listado completo.

4.12 Prueba integral con cinco estudiantes

Finalmente, se probó el sistema completo registrando al menos cinco estudiantes, comprobando el registro, la consulta del JSON individual y general, la edición, la eliminación, la búsqueda, el filtrado por carrera, la limpieza general y la actualización del contador en cada caso.

Total de estudiantes registrados: 5

Buscar por nombre, carrera o semestre

Buscar

Limpiar búsqueda

Tecnologías de la Información



Filtrar

#	Nombre	Carrera	Semestre	Acciones
1	Daniel Barragan	Tecnologías de la Información	4	Ver JSON Editar Eliminar
2	Steven barragan	Software	3	Ver JSON Editar Eliminar
3	Lucia Chisaguano	Tecnologías de la Información	4	Ver JSON Editar Eliminar
4	Andres Benavides	Sistemas	2	Ver JSON Editar Eliminar
5	Jordy Calderon	Software	8	Ver JSON Editar Eliminar

Figura 15. Tabla con al menos cinco estudiantes registrados, utilizada para las pruebas integrales del sistema.

5. LIBRERÍA EXTERNA UTILIZADA

Para la actividad complementaria se integraron las librerías SweetAlert2 y Toastify, ambas incorporadas mediante CDN en index.html. SweetAlert2 se utilizó para reemplazar los cuadros de confirmación básicos del navegador (confirm()) por ventanas modales con mejor diseño visual, empleadas antes de eliminar un estudiante y antes de limpiar toda la información. Toastify se utilizó para mostrar notificaciones breves y no bloqueantes en la esquina superior derecha de la pantalla cada vez que se agrega, edita, elimina o limpia información, sin interrumpir el flujo de uso del formulario.

Código HTML (inclusión por CDN):

```

1. <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/toastify-js/src/toastify.min.css" />
2. ...
3. <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11/dist/sweetalert2.all.min.js"></script>
4. <script src="https://cdn.jsdelivr.net/npm/toastify-js"></script>

```

Frente a los mensajes básicos de alert() y confirm() del navegador, ambas librerías aportan una presentación más profesional, iconografía, colores según el tipo de mensaje (éxito, advertencia, error, información) y una experiencia de usuario más moderna y menos intrusiva, sin bloquear la ejecución del script mientras se muestran.

6. ANÁLISIS DE RESULTADOS

A partir de las pruebas realizadas se verificó que el sistema mantiene la información de los estudiantes de forma centralizada en el arreglo estudiantes, y que tanto la tabla como el contador, el modal de JSON, la búsqueda y el filtro reflejan siempre el estado actual de dicho arreglo. Las operaciones de edición y eliminación funcionan correctamente sobre los índices del arreglo, y la confirmación previa a la eliminación evita pérdidas accidentales de información. La búsqueda y el filtro por carrera operan siempre sobre el arreglo completo de estudiantes y no sobre las filas ya dibujadas, por lo que los resultados mostrados son consistentes incluso al combinarlos con otras acciones del sistema.

7. DISCUSIÓN

El uso de un arreglo de objetos como fuente única de datos, junto con una función central de renderizado (renderTabla), simplifica considerablemente el mantenimiento del sistema: cualquier cambio en los datos (agregar, editar, eliminar, buscar o filtrar) se resuelve modificando el arreglo y volviendo a invocar la misma función de dibujo, en lugar de manipular directamente cada fila de la tabla. La delegación de eventos sobre la tabla evita tener que reasignar listeners cada vez que se regeneran las filas. Como oportunidad de mejora, se identifica que las acciones de edición y eliminación actualmente se basan en el índice del arreglo (data-index), por lo que, al trabajar sobre resultados filtrados o buscados, sería recomendable asignar un identificador único a cada estudiante (por ejemplo mediante Date.now()) para evitar editar o eliminar el registro incorrecto.

8. CONCLUSIONES Y CRITERIO PERSONAL

El desarrollo de esta práctica permitió reforzar el manejo del DOM y de los eventos en JavaScript, comprendiendo cómo un mismo conjunto de datos almacenado en un arreglo de objetos puede reflejarse de manera consistente en una tabla, en un contador, en un modal y en los resultados de una búsqueda o un filtro. Se afianzó también el manejo de tablas dinámicas mediante createElement() y appendChild(), la representación de información en formato JSON con JSON.stringify(), y la implementación de operaciones CRUD completas (creación, lectura, edición y eliminación) sobre datos en memoria. Finalmente, la integración de librerías externas como SweetAlert2 y Toastify permitió comprobar cómo herramientas de terceros pueden mejorar significativamente la experiencia visual de una aplicación web sin añadir demasiada complejidad al código.

9. BIBLIOGRAFÍA

Mozilla Developer Network. 2024. Document Object Model (DOM), MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

Bootstrap Team. 2024. Modal · Bootstrap v5.3, Documentación oficial de Bootstrap. <https://getbootstrap.com/docs/5.3/components/modal/>

SweetAlert2. 2024. SweetAlert2 Documentation. <https://sweetalert2.github.io/>

Apvarun. 2024. Toastify-js Documentation, GitHub. <https://github.com/apvarun/toastify-js>